Step 4 — A Router That Sends Each Message to the Right Agent Automatically — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 4: after Step 3, *you* choose the agent by typing `/extract`. This step hands that choice to a router, gives the Concierge its own file, and moves the "router → agent" flow into a service. That turns two modes of one script into a real 2-agent system. **If not:** users would have to know and type a command to file a ticket — exactly the manual sorting this project exists to remove.

After this step the project is complete. Files from Steps 1-3 that aren't shown here (`config.py`, `exceptions.py`, `models/llm.py`, `agents/triage.py`, `prompts/concierge.txt`, `prompts/triage.txt`, everything in `utils/`, `tests/unit/test_history.py`, `scripts/triage_samples.py`) don't change.

Solution — a model-call router with a logged reason, and a service layer

Changed from Step 3: `SupportTicket` is unchanged; `RouteDecision` is added below it.

Story — `schemas.py`: adds `RouteDecision` next to `SupportTicket`. The router's answer is structured output too, the way Doc04's table suggests for "a supervisor picks the next agent". **If not:** you'd be parsing a free-text reply like "I think triage." by hand, and a typo'd agent name could slip through.

```
# src/schemas.py
# The shapes the model must answer in. SupportTicket: one support ticket.
# RouteDecision: which agent gets a message, and why.
from typing import Literal

from pydantic import BaseModel

class SupportTicket(BaseModel):
    customer_name: str | None
    issue_category: Literal["billing", "technical", "account_access", "other"]
    urgency: Literal["low", "medium", "high"]
    summary: str

# new in Step 4
```

```
class RouteDecision(BaseModel):
    agent: Literal["concierge", "triage"]
    reason: str
```

Story — `prompts/router.txt`: says exactly what counts as "triage" and what counts as "concierge", and asks for a short reason. **If not:** the model would guess its own dividing line, and it would move from message to message.

`src/prompts/router.txt`

```
You route messages for a customer support desk. Choose 'triage' if the
message reports a problem that needs a support ticket (a wrong charge,
something broken, no access to an account). Choose 'concierge' for
greetings, small talk and general questions. Give a one-sentence reason.
```

Story — `agents/router.py`: one small structured call per message, which understands wording no keyword list covers. It starts from a safe default (Concierge) and only replaces it with a usable answer. Every decision is logged with its reason. **If not:** unclear messages would be misrouted with no trace, and a failed routing call could crash the loop or file a false ticket.

```
# src/agents/router.py
# The router: decide() asks the model which agent should answer a message,
# and falls back to the Concierge when the answer can't be used.
from openai import OpenAI
from pydantic import ValidationError

from src.models.llm import MODEL
from src.schemas import RouteDecision
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def decide(client: OpenAI, message: str) -> RouteDecision:
    logger = get_logger(__name__)
    messages = [
        {"role": "system", "content": load_prompt("router")},
        {"role": "user", "content": message},
    ]
    decision = RouteDecision(
        agent="concierge", reason="router could not decide"
    )
    try:
        completion = client.beta.chat.completions.parse(
            model=MODEL, messages=messages, response_format=RouteDecision
        )
        reply = completion.choices[0].message
```

```
        if reply.parsed is not None:
            decision = reply.parsed
    except ValidationError as e:
        logger.warning(f"Router output did not match RouteDecision: {e}")

    logger.info(
        f"routed to {decision.agent} ({decision.reason}): {message}"
    )
    return decision
```

Moved from Step 3's `main.py`: the system-prompt line, `MAX_HISTORY_TOKENS`, and the trim + stream lines of the chat branch. Nothing new is added — they're just given their own file.

Story — `agents/concierge.py`: Step 3's chat branch becomes an agent: its own prompt (`start_history()`), its own token budget, one function to call. **If not:** the "Concierge" would only be a name for some lines inside `main.py`.

```
# src/agents/concierge.py
# The Concierge agent: start_history() begins a chat with its system
# prompt, and concierge_agent() trims the chat and streams the reply.
from openai import OpenAI

from src.models.llm import stream_message
from src.utils.history import trim_history
from src.utils.prompts import load_prompt

MAX_HISTORY_TOKENS = 3000

def start_history() -> list[dict]:
    system_prompt = load_prompt("concierge")
    return [{"role": "system", "content": system_prompt}]

def concierge_agent(client: OpenAI, history: list[dict], message: str) -> str:
    trim_history(history, MAX_HISTORY_TOKENS)
    return stream_message(client, history, message)
```

Changed from Step 3: `print_ticket()` stays exactly as it was; three imports and `handle_message()` are added below it. `handle_message()` holds the old `/extract`-or-chat `if`, but the router makes the choice now.

Story — `services/desk_service.py`: the whole flow in one function: ask the router, run the agent it picked, show the result. **If not:** `main.py` would mix keyboard input with business logic, and Project 5's web API would have to copy this flow instead of calling it.

```
# src/services/desk_service.py
# The app's flow, apart from the keyboard: handle_message() asks the router,
# then calls the chosen agent. print_ticket() shows a ticket.
# new in Step 4: every import except SupportTicket
from openai import OpenAI

from src.agents.concierge import concierge_agent
from src.agents.router import decide
from src.agents.triage import triage_agent
from src.schemas import SupportTicket
from src.utils.logger import get_logger

def print_ticket(ticket: SupportTicket) -> None:
    name = ticket.customer_name
    if name is None:
        name = "(not given)"
    print("Ticket created:")
    print(f"  customer: {name}")
    print(f"  category: {ticket.issue_category}")
    print(f"  urgency:  {ticket.urgency}")
    print(f"  summary:  {ticket.summary}")

# new in Step 4
def handle_message(client: OpenAI, history: list[dict], message: str) -> None:
    logger = get_logger(__name__)
    decision = decide(client, message)

    if decision.agent == "triage":
        ticket = triage_agent(client, message)
        if ticket is None:
            print("Sorry, I couldn't turn that into a ticket — please")
            print("describe the problem again in a little more detail.")
        else:
            logger.info(
                f"ticket: {ticket.issue_category} / {ticket.urgency}"
            )
            print_ticket(ticket)
    else:
        print("Concierge: ", end="", flush=True)
        concierge_agent(client, history, message)
```

Changed from Step 3: the startup and every `except` block are the same. What's gone: the `print_ticket` import, `MAX_HISTORY_TOKENS` and the `if/else` body (they now live in `desk_service.py` and `concierge.py`). What's new: `history = start_history()` and one call, `handle_message(...)`, inside the `try`.

Story — `main.py`: now only startup, the input loop, and Step 3's error handling (unchanged). `/extract` is gone. **If not:** `main.py` would keep growing with every new agent, instead of staying a thin entry point.

```
# main.py
# The file you run: python main.py
# Startup, the input loop and the error messages — every message goes to
# handle_message(), which picks the agent.
import sys

import openai

from src.agents.concierge import start_history
from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import check_api_key_at_startup, create_client
from src.services.desk_service import handle_message
from src.utils.logger import get_logger

def main() -> None:
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    client = create_client(config)
    if not check_api_key_at_startup(client):
        logger.error("Authentication failed — check OPENAI_API_KEY.")
        print("Login failed — check OPENAI_API_KEY in your .env file.")
        sys.exit(1)

    history = start_history()
    print("SupportDesk AI — type a message, or /quit to exit.")

    while True:
        message = input("You: ").strip()
        if message == "/quit":
            break
        if message == "":
            continue

        # changed in Step 4: one call replaces the /extract-or-chat if/else
        try:
            handle_message(client, history, message)
        except openai.AuthenticationError:
```

```
            logger.error("API key stopped working mid-session.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
        except openai.RateLimitError:
            logger.warning("Rate limited after the client's retries.")
            print("We're busy right now — wait a moment and try again.")
        except openai.BadRequestError as e:
            logger.warning(f"Request rejected (likely too long): {e}")
            if history[-1]["role"] == "user":
                history.pop()
            print("That message was too long to handle — please shorten it.")

if __name__ == "__main__":
    main()
```

Changed from Step 3: the `"triage"` list is the same; the `"routing"` list is new.

Story — `data/sample/messages.json`: adds routing cases — each message with the agent it should reach. **If not:** routing could only be checked by chatting by hand.

`data/sample/messages.json`

```
{
  "triage": [
    "Hi, I'm Sarah Khan. I was charged twice this month. Please fix it today.",
    "can't log into my account, tried resetting password twice, still nothing",
    "just wondering what your refund policy is, no rush"
  ],
  "routing": [
    {"message": "hi there, how are you today?", "expected": "concierge"},
    {"message": "what are your support hours?", "expected": "concierge"},
    {
      "message": "I was charged twice this month, please fix it",
      "expected": "triage"
    },
    {
      "message": "the app crashes every time I open settings",
      "expected": "triage"
    }
  ]
}
```

(The first message is wrapped onto 2 lines here only to fit the page — in your file keep it on **one** line, because a JSON string can't contain a line break.)

Changed from Step 3: the import adds `RouteDecision`; cases 1-2 are the same; `test_route_rejects_unknown_agent()` is new.

Story — `tests/unit/test_schemas.py`: one more case — `RouteDecision` rejects an agent name that doesn't exist. **If not:** someone could widen the `Literal` by mistake, and the router could return an agent `handle_message()` doesn't know.

```
# tests/unit/test_schemas.py
# Checks the Pydantic schemas accept good data and reject bad data.
# Each case prints OK or WRONG. No API calls, no API key needed.
# Run from the repo root: python -m tests.unit.test_schemas
from pydantic import ValidationError

from src.schemas import RouteDecision, SupportTicket


def test_ticket_without_a_name() -> None:
    ticket = SupportTicket(
        customer_name=None,
        issue_category="account_access",
        urgency="medium",
        summary="Customer can't log in after two password resets.",
    )
    result = "OK"
    if ticket.customer_name is not None:
        result = "WRONG"
    print(f"case 1 (no name allowed)  -> {result}")


def test_ticket_rejects_unknown_urgency() -> None:
    result = "WRONG"
    try:
        SupportTicket(
            customer_name="Sarah Khan",
            issue_category="billing",
            urgency="urgent",
            summary="Charged twice.",
        )
    except ValidationError:
        result = "OK"
    print(f"case 2 (bad urgency)      -> {result}")


# new in Step 4
def test_route_rejects_unknown_agent() -> None:
    result = "WRONG"
    try:
```

```
        RouteDecision(agent="billing", reason="looks like billing")
    except ValidationError:
        result = "OK"
    print(f"case 3 (bad agent name)   -> {result}")


if __name__ == "__main__":
    test_ticket_without_a_name()
    test_ticket_rejects_unknown_urgency()
    # new in Step 4
    test_route_rejects_unknown_agent()
```

Changed from Step 3: one new import (`decide`), one new function (`test_routing()`), and one new call at the bottom. Cases 1-5 are the same.

Story — `tests/integration/test_live_agents.py` : Step 3's checks plus case 6 — every routing case must reach its expected agent. **If not:** a small change to `router.txt` could quietly send complaints to the Concierge.

```
# tests/integration/test_live_agents.py
# Checks the app against the real OpenAI API — each step adds cases.
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents
import json

from openai import OpenAI

# new in Step 4
from src.agents.router import decide
from src.agents.triage import triage_agent
from src.config import load_config
from src.models.llm import (
    check_api_key_at_startup,
    create_client,
    send_message,
)

SAMPLES_PATH = "data/sample/messages.json"


def test_first_reply(client: OpenAI) -> None:
    reply = send_message(client, [], "What's 2+2?")
    result = "OK"
    if "4" not in reply:
        result = "WRONG"
    print(f"case 1 (first reply) -> {result}: {reply}")
```

```
def test_bad_key() -> None:
    bad_client = OpenAI(api_key="sk-fake-key-123")
    result = "OK"
    if check_api_key_at_startup(bad_client):
        result = "WRONG"
    print(f"case 2 (bad key)     -> {result}")


def test_memory(client: OpenAI) -> None:
    history = [{"role": "system", "content": "You are a helpful assistant."}]
    send_message(client, history, "My order number is 4471.")
    send_message(client, history, "Thanks, that's all for now.")
    reply = send_message(client, history, "What was my order number?")
    result = "OK"
    if "4471" not in reply:
        result = "WRONG"
    print(f"case 3 (memory)      -> {result}: {reply}")


def load_samples() -> dict:
    with open(SAMPLES_PATH) as samples_file:
        return json.loads(samples_file.read())


def test_every_sample_gets_a_ticket(client: OpenAI, samples: dict) -> None:
    for message in samples["triage"]:
        ticket = triage_agent(client, message)
        if ticket is None:
            print("case 4 (ticket)      -> WRONG: no ticket")
        else:
            print("case 4 (ticket)      -> OK:", ticket.issue_category)


def test_no_invented_name(client: OpenAI, samples: dict) -> None:
    ticket = triage_agent(client, samples["triage"][1])
    result = "OK"
    if ticket is None or ticket.customer_name is not None:
        result = "WRONG"
    print(f"case 5 (no name)     -> {result}")


# new in Step 4
def test_routing(client: OpenAI, samples: dict) -> None:
    for case in samples["routing"]:
        decision = decide(client, case["message"])
```

```
        result = "OK"
        if decision.agent != case["expected"]:
            result = "WRONG"
        print(f"case 6 (routing)     -> {result}: {decision.agent}")


if __name__ == "__main__":
    config = load_config()
    client = create_client(config)
    samples = load_samples()
    test_first_reply(client)
    test_bad_key()
    test_memory(client)
    test_every_sample_gets_a_ticket(client, samples)
    test_no_invented_name(client, samples)
    # new in Step 4
    test_routing(client, samples)
```

Run everything (from the repo root):

```
python -m tests.unit.test_config
python -m tests.unit.test_history
python -m tests.unit.test_schemas
python -m tests.integration.test_live_agents
python main.py
```

Expected output — `test_schemas` gets one more line, and `test_live_agents` ends with four routing lines:

```
case 3 (bad agent name)   -> OK
```

```
case 6 (routing)     -> OK: concierge
case 6 (routing)     -> OK: concierge
case 6 (routing)     -> OK: triage
case 6 (routing)     -> OK: triage
```

Expected app session (the log lines go to the console too; the model's wording varies):

```
SupportDesk AI — type a message, or /quit to exit.
You: hi! what are your support hours?
... src.agents.router INFO routed to concierge (The user asks
a general question about support hours.): hi! what are your support
hours?
Concierge: We're available Monday to Friday, 9am to 6pm.
```

```
You: can't log into my account, tried resetting password twice
... src.agents.router INFO routed to triage (The user cannot
access their account.): can't log into my account, tried ...
... src.services.desk_service INFO ticket: account_access / medium
Ticket created:
  customer: (not given)
  category: account_access
  urgency:  medium
  summary:  Customer can't log in even after resetting the password twice.
You: /quit
```

(Log lines are shortened with `...` and wrapped to fit the page.)

Now try an unclear one, like `"just wondering what your refund policy is, no rush"`, and read the log's reason. Whichever agent it picks, you can see *why* — that's what the checklist's "logged, with the reason" means.